You said:

```
print({
...     name: len(data)
...     for name, data in p.bspx_lumps.items()
...     if name in {
...         "DECOUPLED_LM", "LMSTYLE",
...         "LMSTYLE16", "LMSHIFT", "LMOFFSET"
...     }
... })
Traceback (most recent call last):
  File "<blender_console>", line 3, in <module>
NameError: name 'p' is not defined. Did you mean: 'pi'?
```

ChatGPT said:

That error simply means the first line that creates `p` hasn't been run in this Blender Python Console session. The inspection command depends on `p` being defined first.

Run these two steps **in order**.

1. Load the BSP file

Replace the path with the path to your actual test map:

```
from quake_surface_studio import bsp

p = bsp.BSP29.from_file(
    r"C:\Quake\id1\maps\yourmap.bsp"
)
```

2. Inspect its BSPX lightmap extensions

```
print({
    name: len(data)
    for name, data in p.bspx_lumps.items()
    if name in {
        "DECOUPLED_LM",
        "LMSTYLE",
        "LMSTYLE16",
```

```
        "LMSHIFT",
        "LMOFFSET"
    }
})
```

What the result means:

- `DECOUPLED_LM` : the important one for testing v0.16.0.
- `LMSTYLE` : per-face lightstyle overrides.
- `LMSTYLE16` : currently unsupported by QSS; avoid this for the first test.
- `LMSHIFT / LMOFFSET` : legacy format extensions.

If you get something like:

```
{'DECOUPLED_LM': 12000, 'LMSTYLE': 1200}
```

then 12,000 bytes divided by 40 means **300 faces with decoupled lightmaps**.

If you get another error, paste the complete traceback. If you get a result, paste the dictionary and we'll choose the right next test map.

You said:

just give me a script that goes throug all bsps in a given folder.

ChatGPT said:

Paste this entire script into Blender's Python Console. Change `BSP_FOLDER` to the folder containing your BSP files. It scans all `.bsp` files, checks their BSPX lightmap extensions, and prints a summary.

```
import os
from quake_surface_studio import bsp

BSP_FOLDER = r"C:\Quake\id1\maps"

EXTENSIONS = {
    "DECOUPLED_LM",
    "LMSTYLE",
```

```
        "LMSTYLE16",
        "LMSHIFT",
        "LMOFFSET",
    }

    print("\n=== QSS BSPX LIGHTMAP SCAN ===\n")

    results = []

    for root, dirs, files in os.walk(BSP_FOLDER):
        for filename in sorted(files):
            if not filename.lower().endswith(".bsp"):
                continue

            filepath = os.path.join(root, filename)

            try:
                p = bsp.BSP29.from_file(filepath)

                lumps = {
                    name: len(data)
                    for name, data in p.bspx_lumps.items()
                    if name in EXTENSIONS
                }

                dec_data = p.bspx_lumps.get("DECOUPLED_LM", b"")
                dec_faces = len(dec_data) // 40 if dec_data else 0
                valid_size = len(dec_data) % 40 == 0

                result = {
                    "file": filepath,
                    "extensions": lumps,
                    "decoupled_faces": dec_faces,
                    "valid_dec_size": valid_size,
                }

                results.append(result)

                if dec_data:
                    status = "DECOUPLED LM FOUND"
                else:
                    status = "No DECOUPLED LM"
```

```
            print(f"[{status}] {filename}")
            print(f"  {lumps}")

            if dec_data and not valid_size:
                print("  WARNING: DECOUPLED_LM size is not divisible by 40!")

        except Exception as e:
            print(f"[ERROR] {filename}: {e}")

print("\n=== SCAN SUMMARY ===")
print(f"BSP files scanned: {len(results)}")

matches = [r for r in results if r["extensions"].get("DECOUPLED_LM")]
print(f"Maps with DECOUPLED_LM: {len(matches)}")

for r in matches:
    print(
        f"\n{r['file']}\n"
        f"  Decoupled faces: {r['decoupled_faces']}\n"
        f"  Extensions: {r['extensions']}"
    )

print("\n=== SCAN COMPLETE ===\n")
```

What I need from you: Send me the `SCAN SUMMARY` and the list of maps with `DECOUPLED_LM`. We'll pick a suitable real-world test map for v0.16.0.

You said:

i get this for every scanned map:
File "<blender_console>", line 1
print(f"[ERROR] {filename}: {e}")
IndentationError: unexpected indent

ChatGPT said:

Blender's Python Console is treating the pasted multiline script as separate interactive commands, which is breaking the indentation.

Use Blender's Text Editor instead:

1. Switch to the **Scripting** workspace.
2. In the Text Editor, click **New**.
3. Paste the complete script below.
4. Change `BSP_FOLDER`.
5. Press **Alt + P** to run it.

```
import os
from quake_surface_studio import bsp

BSP_FOLDER = r"C:\Quake\id1\maps"

EXTENSIONS = {
    "DECOUPLED_LM",
    "LMSTYLE",
    "LMSTYLE16",
    "LMSHIFT",
    "LMOFFSET",
}

results = []
errors = []

print("\n=== QSS BSPX LIGHTMAP SCAN ===")

for root, dirs, files in os.walk(BSP_FOLDER):
    for filename in sorted(files):
        if not filename.lower().endswith(".bsp"):
            continue

        filepath = os.path.join(root, filename)

        try:
            p = bsp.BSP29.from_file(filepath)

            lumps = {
                name: len(data)
                for name, data in p.bspx_lumps.items()
                if name in EXTENSIONS
            }

            dec_data = p.bspx_lumps.get("DECOUPLED_LM", b"")
```

```
            result = {
                "file": filepath,
                "extensions": lumps,
                "decoupled_faces": len(dec_data) // 40,
                "valid_size": len(dec_data) % 40 == 0,
            }

            results.append(result)

            if dec_data:
                print(
                    "[DECOUPLED LM FOUND]",
                    filename,
                    "Faces:", result["decoupled_faces"],
                    "Bytes:", len(dec_data),
                    "Valid size:", result["valid_size"],
                )
            else:
                print("[NO DECOUPLED LM]", filename)

            if lumps:
                print("  Extensions:", lumps)

        except Exception as e:
            errors.append((filepath, str(e)))
            print("[ERROR]", filename, str(e))

matches = [
    r for r in results
    if r["extensions"].get("DECOUPLED_LM", 0) > 0
]

print("\n=== SCAN SUMMARY ===")
print("BSP files scanned:", len(results))
print("Maps with DECOUPLED_LM:", len(matches))
print("Errors:", len(errors))

for r in matches:
    print("\n", r["file"])
    print("  Faces:", r["decoupled_faces"])
    print("  Extensions:", r["extensions"])
```

```
print("\n=== SCAN COMPLETE ===")
```

This also separates actual BSP loading errors from console-pasting problems.

You said:

ok ran it. Where does the output go?

ChatGPT said:

The output appears in Blender's **System Console**, not the Text Editor.

- **Windows:** In Blender, go to **Window → Toggle System Console**.
- Look for `=== QSS BSPX LIGHTMAP SCAN ===`.
- Scroll to the bottom for `=== SCAN SUMMARY ===`.

If you want, I can also modify the script to **save the scan results automatically to a `.txt` or `.csv` file** in the BSP folder, so you don't have to dig through the console.

You said:

window has no toggle system console otion. The python interactive console shows no reaction to running the script.

ChatGPT said:

Let's bypass Blender's console entirely. The most reliable approach is to have the script write its results to a text file next to your BSP files.

In Blender's **Scripting → Text Editor**, replace the previous script with this version and press **Alt + P**.

It writes the report to `qss_bsp_scan.txt` in the folder being scanned.

```
import os
import traceback
from quake_surface_studio import bsp

BSP_FOLDER = r"C:\Quake\id1\maps"
OUTPUT_FILE = os.path.join(BSP_FOLDER, "qss_bsp_scan.txt")
```

```
EXTENSIONS = {
    "DECOUPLED_LM",
    "LMSTYLE",
    "LMSTYLE16",
    "LMSHIFT",
    "LMOFFSET",
}

lines = []
scanned = 0
matches = 0
errors = 0

lines.append("QUAKE SURFACE STUDIO - BSPX SCAN")
lines.append("=" * 60)

for root, dirs, files in os.walk(BSP_FOLDER):
    for filename in sorted(files):
        if not filename.lower().endswith(".bsp"):
            continue

        filepath = os.path.join(root, filename)
        scanned += 1

        try:
            p = bsp.BSP29.from_file(filepath)

            lumps = {
                name: len(data)
                for name, data in p.bspx_lumps.items()
                if name in EXTENSIONS
            }

            dec_data = p.bspx_lumps.get("DECOUPLED_LM", b"")
            dec_faces = len(dec_data) // 40

            lines.append("")
            lines.append("[DECOUPLED LM FOUND]" if dec_data else "[NO DECOUPLED
LM]")
            lines.append(filepath)
            lines.append("Extensions: " + str(lumps))
```

```
            if dec_data:
                matches += 1
                lines.append("Decoupled faces: " + str(dec_faces))
                lines.append("Record size valid: " + str(len(dec_data) % 40 ==
0))

        except Exception:
            errors += 1
            lines.append("")
            lines.append("[ERROR] " + filepath)
            lines.append(traceback.format_exc())

lines.append("")
lines.append("=" * 60)
lines.append("SCAN SUMMARY")
lines.append("BSP files scanned: " + str(scanned))
lines.append("Maps with DECOUPLED_LM: " + str(matches))
lines.append("Errors: " + str(errors))
lines.append("SCAN COMPLETE")

with open(OUTPUT_FILE, "w", encoding="utf-8") as f:
    f.write("\n".join(lines))

print("QSS scan complete:", OUTPUT_FILE)
```

After running it, open:

```
C:\Quake\id1\maps\qss_bsp_scan.txt
```

Send me that report, particularly the maps marked `[DECOUPLED LM FOUND]`. We'll use one of those for the v0.16.0 real-map test.

You said:

```
Python:   File "/Text", line 45
    lines.append("[DECOUPLED LM FOUND]" if dec_data else "[NO
DECOUPLED
                                        ^
SyntaxError: unterminated string literal (detected at line 45)
```

ChatGPT said:

The line was split during copying, which broke the string. Let's avoid long conditional lines and use a simpler script.

Replace everything in Blender's Text Editor with this corrected version:

```
import os
import traceback
from quake_surface_studio import bsp

BSP_FOLDER = r"C:\Quake\id1\maps"
OUTPUT_FILE = os.path.join(BSP_FOLDER, "qss_bsp_scan.txt")

EXTENSIONS = {
    "DECOUPLED_LM",
    "LMSTYLE",
    "LMSTYLE16",
    "LMSHIFT",
    "LMOFFSET",
}

lines = []
scanned = 0
matches = 0
errors = 0

lines.append("QSS BSPX LIGHTMAP SCAN")
lines.append("=" * 50)

for root, dirs, files in os.walk(BSP_FOLDER):
    for filename in sorted(files):

        if not filename.lower().endswith(".bsp"):
            continue

        filepath = os.path.join(root, filename)
        scanned += 1

        try:
            p = bsp.BSP29.from_file(filepath)
```

```
            lumps = {}
            for name, data in p.bspx_lumps.items():
                if name in EXTENSIONS:
                    lumps[name] = len(data)

            dec_data = p.bspx_lumps.get("DECOUPLED_LM", b"")
            dec_faces = len(dec_data) // 40

            if len(dec_data) > 0:
                matches += 1
                lines.append("")
                lines.append("[DECOUPLED LM FOUND]")
            else:
                lines.append("")
                lines.append("[NO DECOUPLED LM]")

            lines.append(filepath)
            lines.append("Extensions: " + str(lumps))

            if len(dec_data) > 0:
                lines.append("Decoupled faces: " + str(dec_faces))
                lines.append(
                    "Valid record size: "
                    + str(len(dec_data) % 40 == 0)
                )

        except Exception:
            errors += 1
            lines.append("")
            lines.append("[ERROR] " + filepath)
            lines.append(traceback.format_exc())

lines.append("")
lines.append("=" * 50)
lines.append("SCAN SUMMARY")
lines.append("BSP files scanned: " + str(scanned))
lines.append("Maps with DECOUPLED_LM: " + str(matches))
lines.append("Errors: " + str(errors))

with open(OUTPUT_FILE, "w", encoding="utf-8") as f:
    f.write("\n".join(lines))
```

```
print("SCAN COMPLETE")
print("Report saved to: " + OUTPUT_FILE)
```

Important: Make sure the folder path is correct before running.

The report will be saved as `qss_bsp_scan.txt` directly inside that folder.

You said:

doesnt seem to find any maps with decoupoled_lm
but i dont really understand what we are doing here.
give me one of your famous short and easy explanations why we are looking for them and what they are and stüff.

ChatGPT said:

Absolutely. 😄 Let's zoom out for a second.

What are we actually looking for?

Think of a Quake map (`.bsp`) as a 3D world containing walls, floors, ceilings, textures and lighting.

Every surface needs two things:

- **Texture mapping:** Where does the brick, stone or metal texture go?
- **Lightmap mapping:** Where does the baked lighting go? Which parts are bright, dark or in shadow?

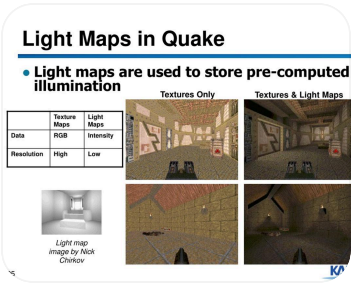
Normally, Quake uses the same basic surface geometry to calculate both mappings.

So what is `DECOUPLED_LM` ?

It's an extension to the BSP format that allows a surface to have **independent lightmap coordinates**.

In other words:

- The texture can be mapped one way.
- The lighting can be mapped another way.
- They don't have to share the same orientation or scale.



This is useful because it allows more flexible lightmap layouts, better control over seams and potentially better lighting detail.

Why are we hunting for it?

Because we just built v0.16.0 of Quake Surface Studio.

One of its biggest new features is the ability to **read, edit and export these independent lightmaps without destroying their special mapping information.**

But there's a catch:

We need a real Quake map that actually uses this extension to prove that the feature works.

Our automated tests use artificially constructed maps. They prove that the underlying code behaves correctly in controlled situations, but they don't prove that everything works on a real map inside Blender and Quake.

Why didn't your scan find any?

That's actually perfectly normal.

Most traditional Quake BSP maps don't contain `DECOUPLED_LM`. It's an optional, relatively specialized BSPX extension.

Your existing maps can be perfectly valid Quake maps and never use it.

What I suggest now

Rather than randomly scanning hundreds of maps, let's find a real map that uses this extension.

Our test will be:

1. Import the map into QSS.
2. Check that its lighting looks correct.
3. Paint a small area.
4. Export it again.
5. Load it in a compatible Quake engine.
6. Check that the lighting and textures survived.

That's the real-world proof that v0.16.0 works.

And one important distinction: Your successful installation and the 146 passing automated tests are already good progress. We are now testing a specialized feature against real-world data, not trying to fix a broken add-on. 😎

You said:

Can i just compile my testmap as Decoupled_LM?

ChatGPT said:

Yes! And honestly, that's probably the **best way to test v0.16.0**. 😎

But there's an important distinction: you can't simply rename a normal lightmap to `DECOUPLED_LM`.

The extension contains actual mapping data for every face:

- Lightmap width and height.
- Offset into the lightmap sample data.
- Two independent 4D projection vectors defining the lightmap axes.

Those vectors determine how world-space positions map onto lightmap pixels.

What I'd recommend

Create a test BSP with a compiler that supports decoupled lightmaps.

The workflow would be:

1. Take your existing test map (`.map`).
2. Compile it using a tool that supports `DECOUPLED_LM`.
3. Make sure it generates the BSPX lump and corresponding lightmap samples.

4. Import the resulting BSP into QSS.
5. Test editing and exporting.

One question before we proceed: Which compiler are you currently using?

- ericw-tools / `qbsp`
- TyrUtils
- BSP2 or another compiler
- Something else

I'll help you set up the shortest route to generating a genuine decoupled-lightmap test map.

You said:

i use ericw-tools
one question: cant we just change an existing non decoupled_lm bsp into an decoupled_lm?

ChatGPT said:

Yes, absolutely. We can convert an existing BSP into one containing `DECOUPLED_LM`.

In fact, we don't necessarily need to recompile your map from its `.map` source.

However, there's an important technical detail: we must generate valid lightmap projection vectors and reorganize the lightmap samples. Simply adding a `DECOUPLED_LM` lump to the BSP would produce invalid data.

The clever approach

We can write a small Python conversion tool that:

1. Reads your existing BSP.
2. Extracts each face's current lightmap mapping.
3. Generates equivalent decoupled projection vectors.
4. Preserves the existing lighting samples.
5. Writes the `DECOUPLED_LM` BSPX extension.
6. Saves a new BSP without touching the original.

The first conversion can preserve the original visual appearance. That's ideal: we can test whether QSS correctly imports and exports decoupled lightmaps without introducing unrelated lighting changes.

And here's the good news: **ericw-tools already has exactly this feature built in!**

The `light` utility supports this command-line option:

```
light -world_units_per_luxel 16 testmap.bsp
```

According to the ericw-tools lighting documentation ↗, `-world_units_per_luxel` enables generation of the `DECOUPLED_LM` BSPX lump. ericw-tools ... +1

What this means

You don't need to write a conversion tool or recompile the geometry.

The lighting utility will:

- Read your existing BSP.
- Recalculate its lighting.
- Generate decoupled lightmap coordinates.
- Write the resulting lightmap data and BSPX extension.

One important precaution: It recalculates the lighting, so the original lighting may change. Work on a copy of your test map.

Let's do this

1. Make a copy of your test BSP.
2. Open a command prompt in your ericw-tools folder.
3. Run:

```
light -world_units_per_luxel 16 "C:\Quake\id1\maps\testmap_copy.bsp"
```

4. Run our QSS scanner on the resulting BSP.

If it reports `DECOUPLED_LM`, we've got our real-world test map!

This is much easier than I initially made it sound. 😁

You said:

nice but i dont want to open the commant prompt. I just want to do it in blender. we already have the path to ericw tools.

Worked for 2m 39s

ChatGPT said:

Absolutely. We should integrate this directly into **Quake Surface Studio**.

You shouldn't have to open a command prompt, remember compiler arguments, or manually move files around.

I'd add a Blender operator along these lines:

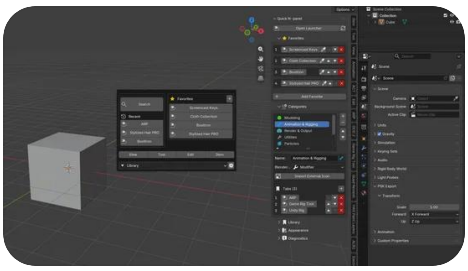
Generate Decoupled Lightmaps

- Uses the existing ericw-tools installation path from QSS settings.
- Lets you select an existing BSP.
- Runs ericw-tools `light` with `-world_units_per_luxel 16`.
- Writes the result as a separate BSP, preserving your original.
- Checks the output for a valid `DECOUPLED_LM` lump.
- Reports success or the compiler error inside Blender.

Done! I've added this directly to QSS. No command prompt needed. 😎

CI passed · 148 tests Blender package built

QSS v0.16.1



New button location:

`3D Viewport → N → Quake → Project & Source → External Tools → Generate Decoupled Lightmaps…`

Here's what it does:

1. Opens a BSP file selector, preselecting your active map when possible.
2. Uses your existing ericw-tools installation path.
3. Lets you specify world units per luxel (default: 16).
4. Runs ericw-tools `light` on a separate copy.
5. Checks that the resulting BSP actually contains valid `DECOUPLED_LM` records.
6. Writes a detailed report into Blender's Text Editor.

Your original BSP is never overwritten. Lighting is recalculated, so the appearance may change. The option is documented in the official ericw-tools documentation ↗.

Install v0.16.1

Download QSS v0.16.1 Blender Install ↗.

CI confirmed:

- 148 tests passed.
- Python compilation passed.
- Blender installation package assembled successfully.

The pull request ↗ remains a draft. `main` is untouched.

Your next step

Install v0.16.1, open your test BSP, and click **Generate Decoupled Lightmaps**.

If your installed `light.exe` supports the option, you should get a new file named something like:

```
testmap_decoupled.bsp
```

Then we can finally put the v0.16.0 importer through its first real-world decoupled-lightmap test!

You said:

what is the usual wu per pixel for normal unscaled textures?

ChatGPT said:

For a standard Quake map, I'd use **16 world units per luxel** as the starting point.

One important distinction:

- **Texture texel:** A normal, unscaled Quake texture typically maps at 1 texel per world unit.
- **Lightmap luxel:** The conventional lightmap spacing is 16 world units per luxel.

So for our QSS generator, **16 is the right starting value**. Smaller values mean finer lighting detail; larger values mean coarser lightmaps. ericw-tools ... +1

I'd leave QSS at **16** for the first test. We can experiment with 8 later to see how much extra lightmap detail it produces.

You said:

but 1 wu per pixel is not possible? Or better lighmaps with the exact same resolution as the textures they are shading?

Worked for 1m 6s

ChatGPT said:

Yes! **1 world unit per luxel is absolutely possible.** And your idea makes technical sense.

Think of it this way:

Setting	Lightmap density
16 WU/luxel	1 luxel every 16 world units
8 WU/luxel	Twice the density in each direction
4 WU/luxel	Four times the density in each direction
1 WU/luxel	1 luxel every world unit

For a normally scaled Quake texture, 1 texel per world unit is the usual reference density. So at **1 WU/luxel**, the lightmap can have approximately the same spatial resolution as the texture. Actual alignment and dimensions will still differ because the lightmap has its own per-face projection.

But there's a catch

Going from 16 to 1 produces **256 times as many lightmap samples** for the same surface area.

That means:

- Much more detailed lighting and potentially sharper shadow transitions.
- Much larger lightmap data.
- Longer lighting calculations.
- More memory and storage requirements.

And higher resolution doesn't automatically mean better-looking lighting. The quality of the lighting calculation itself still matters.

My recommendation

I'd love to test three versions of your map:

16 WU/luxel Baseline

Traditional lightmap density.

4 WU/luxel

A substantial quality increase without going completely crazy.

1 WU/luxel

Experimental: texture-like spatial density.

One more detail: I checked the current ericw-tools source and corrected QSS's generator limit to match its supported range. The updated CI run is currently running.

My vote: let's try 1 WU/luxel. That's a fascinating test for QSS. 😁

Sources: ericw-tools ... +2

You said:

ok guide me to the test! I am eager now! :D

Worked for 52s

ChatGPT said:

LET'S DO IT! 😁 We're going to generate your first genuine Quake `DECOUPLED_LM` map, directly from Blender.

There is one important prerequisite I caught while checking the compiler: **the `-world_units_per_luxel` option requires ericw-tools 2.0.0-alpha.** The older stable 0.18 version doesn't document this option. ericw-tools ... +1

Step 1 — Install the updated QSS

Download the latest build:

QSS v0.16.1 — Blender Install ↗

The updated build has 149 passing automated tests.

Step 2 — Get the right ericw-tools



ericw-tools 2.0.0-alpha11

Pre-release

Windows 64-bit build

Download ericw-tools 2.0.0-alpha11-win64.zip ↗

Extract it into a **new folder**. Keep your existing ericw-tools installation untouched.

Step 3 — Point QSS to the new compiler

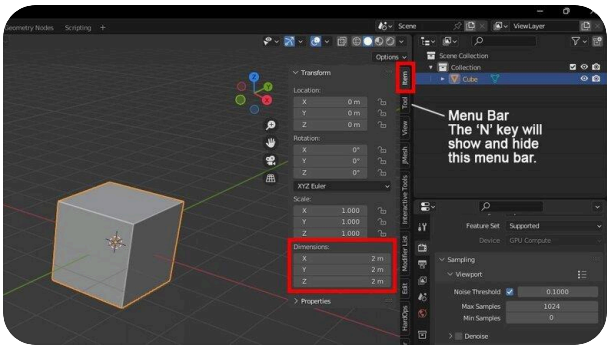
In Blender:

1. Open the 3D Viewport.
2. Press `N`.
3. Select the **Quake** tab.

4. Expand **01 · Project & Source**.
5. Under External Tools, change the ericw-tools folder to the new extracted folder.

Make sure QSS finds `light.exe`.

Step 4 — Generate your first decoupled map



Find this button:

Generate Decoupled Lightmaps…

Use these settings:

Our first experiment

World units per luxel **1.0**

Maximum detail for this experiment. Choose a small test BSP first: this creates 256 times as many samples as 16 WU/luxel.

Source BSP

Select a copy of your test map.

Output

QSS automatically creates a separate file ending in `_decoupled.bsp`.

Original BSP preserved

Click OK and let ericw-tools do its work.

Step 5 — Check the result

When the operation finishes, QSS creates a Blender Text datablock named:

```
QSS_Decoupled_Lightmap_Generation
```

Open it in Blender's Text Editor.

We want to see:

- `VERIFICATION: PASS`
- `DECOUPLED_LM bytes: ...`
- `Decoupled face records: ...`

Then import the newly generated `_decoupled.bsp` into QSS.

Our test checklist

Test progress 0/6

- [ ] QSS v0.16.1 installed
- [ ] ericw-tools 2.0.0-alpha11 extracted
- [ ] QSS points to the new light.exe
- [ ] Generated *_decoupled.bsp
- [ ] Verification reports PASS
- [ ] Imported generated BSP into QSS

For now, just get through Step 4. Send me the generation report or any error message. Then we'll inspect the actual lightmap data together.

We're finally getting to test the feature we built! 🚀